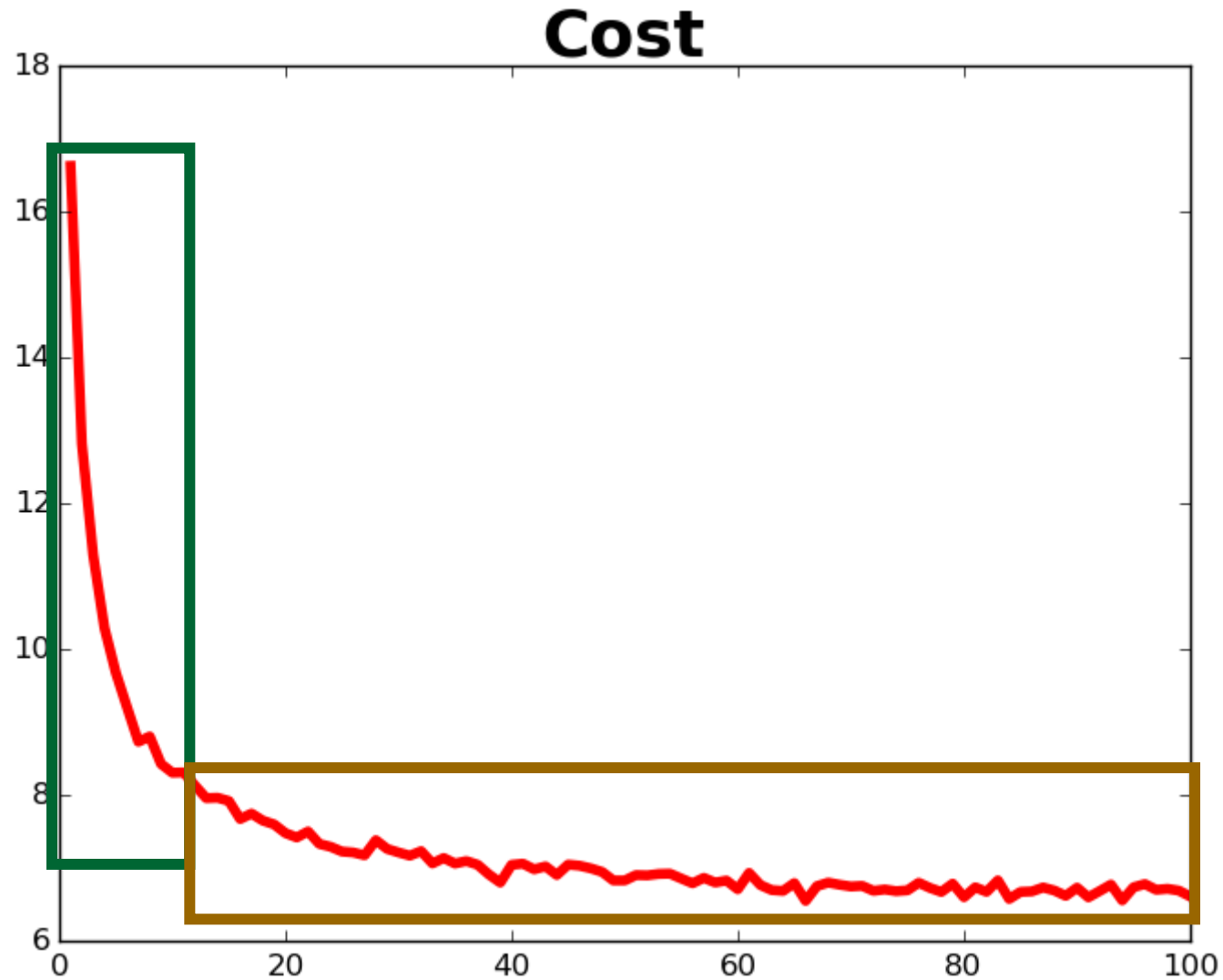


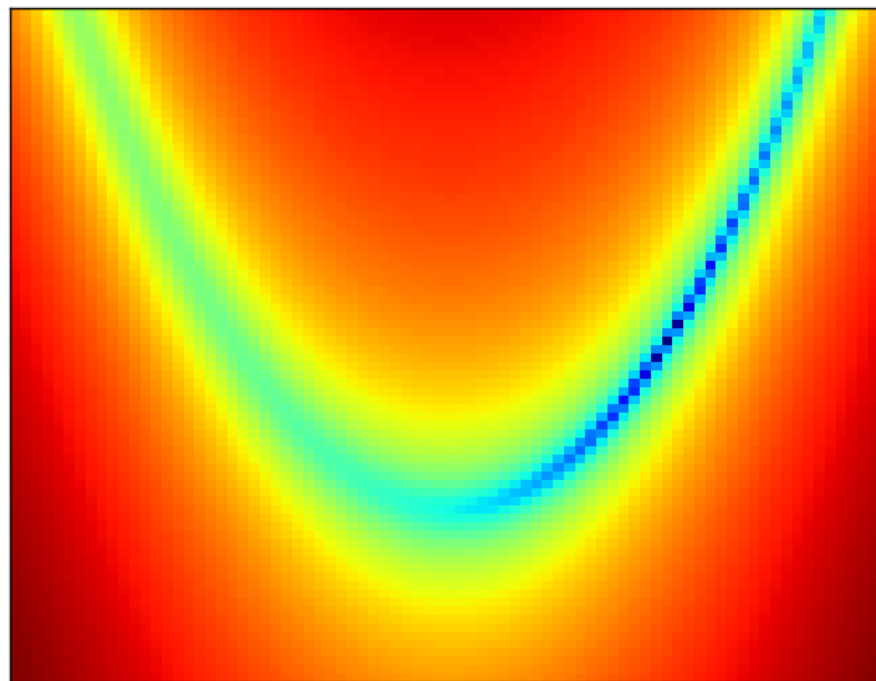
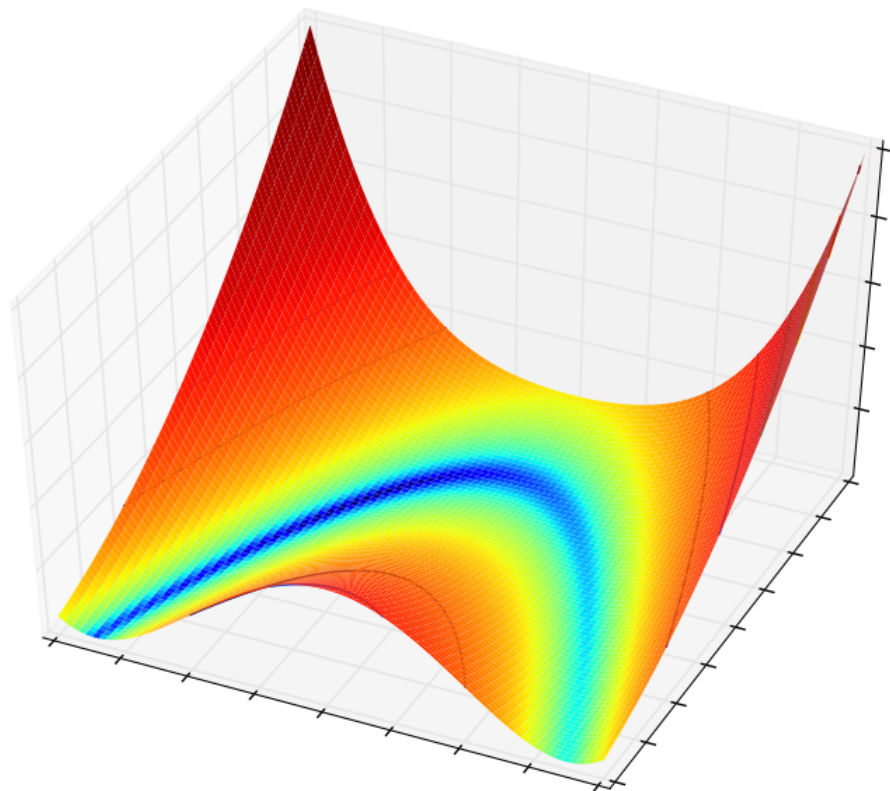
Sieci Neuronowe i Uczenie Głębokie

Aspekty optymalizacyjne w uczeniu sieci neuronowych

Dwie fazy uczenia sieci neuronowej



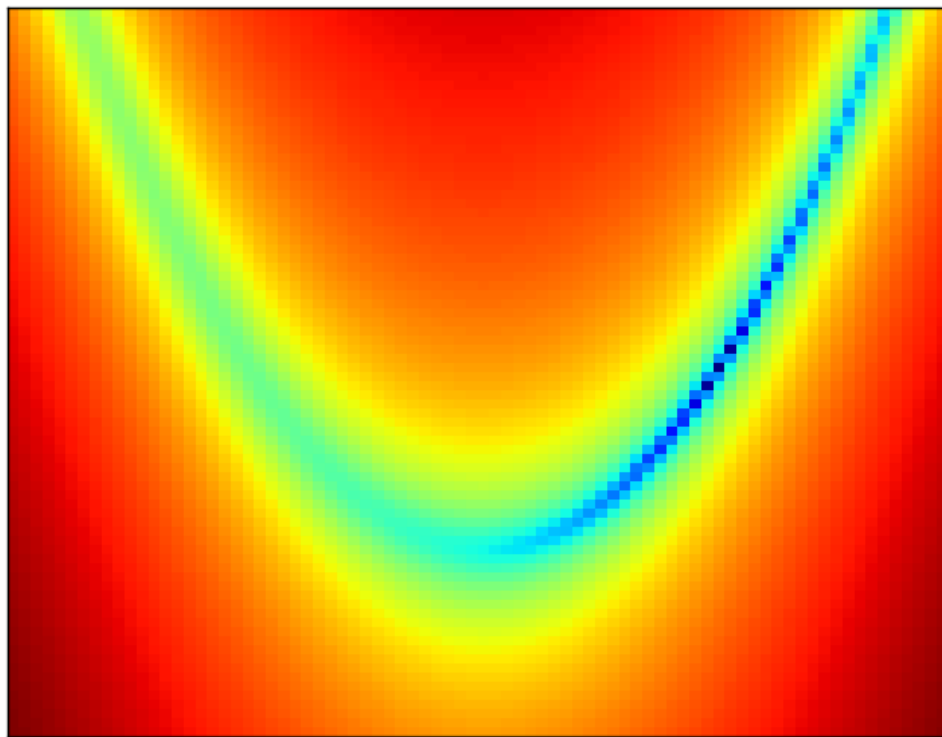
Dwie fazy uczenia sieci neuronowej



Po wstępnej fazie uczenia, funkcja kosztu często przybiera postać prawie płaskiej „doliny” (ze stromymi zboczami).

Dwie fazy uczenia sieci neuronowej

Jak przyspieszyć minimalizację takiej funkcji kosztu?



- ❑ Dotychczas na wykładzie przedstawiona była metoda pędu.
- ❑ Jednak pomimo optymalizacji z pędem, trenowane modele często nie osiągały wysokiej skuteczności.
- ❑ Problemem był zanikający gradient, związany z niewielkimi początkowymi wagami lub z wysyceniem funkcji aktywacji.
- ❑ Konieczny był pretrening lub modyfikacja funkcji aktywacji (np. ReLU) i odpowiednia regularyzacja (np. Dropout).

Metoda Nesterova

Czy można poprawić skuteczność sieci neuronowej wykorzystując mocniejszą metodę optymalizacyjną do minimalizacji funkcji kosztu?

Tak:

I. Sutskever, J. Martens, G. Dahl, G.E. Hinton. "On the importance of momentum and initialization in deep learning", Proc. 30th ICML, 2013

W powyższej pracy przedstawiono modyfikację klasycznej metody pędu, pozwalającą uczyć sieci neuronowe z dużym pędem.

- Zaproponowana modyfikacja realizuje w istocie jedną z metod tzw. przyspieszonego spadku wzdłuż gradientu: **metodę Nesterova**.

Metoda Nesterova

Y. Nesterov. "A method of solving a convex programming problem with convergence rate $O(1/k^2)$ ", Soviet Mathematics Doklady, 27, pp. 372–376, 1983.

Przyspieszony spadek wzdłuż gradientu (*Nesterov's Accelerated Gradient*):

- Modyfikacja klasycznej metody spadku wzdłuż gradientu – a więc metoda pierwszego rzędu.
- Zbiega asymptotycznie szybciej, niż klasyczny spadek wzdłuż gradientu.
 - Przy pewnych założeniach, w metodzie Nesterova błąd minimalizacji spada z liczbą kroków k jak $O(1/k^2)$.
 - Dla spadku wzdłuż gradientu będzie to odpowiednio $O(1/k)$.

Metoda Nesterova

Metodę Nesterova można wyrazić jako zmodyfikowaną metodę pędu (*I. Sutskever et al., Proc. ICML 2013*).

Zmieniona jest poprawka do macierzy pędu.

- W klasycznej metodzie pędu:

$$\mathbf{M}_t = \mu \mathbf{M}_{t-1} - \epsilon \nabla_{\phi_t} J$$

- W metodzie Nesterova:

$$\mathbf{M}_t = \mu \mathbf{M}_{t-1} - \epsilon \nabla_{\phi_t + \mu \mathbf{M}_{t-1}} J$$

W metodzie Nesterova gradient jest więc wyznaczamy nie dla aktualnych parametrów modelu, lecz dla parametrów do których prowadzi nas pęd z poprzedniego kroku uczącego.

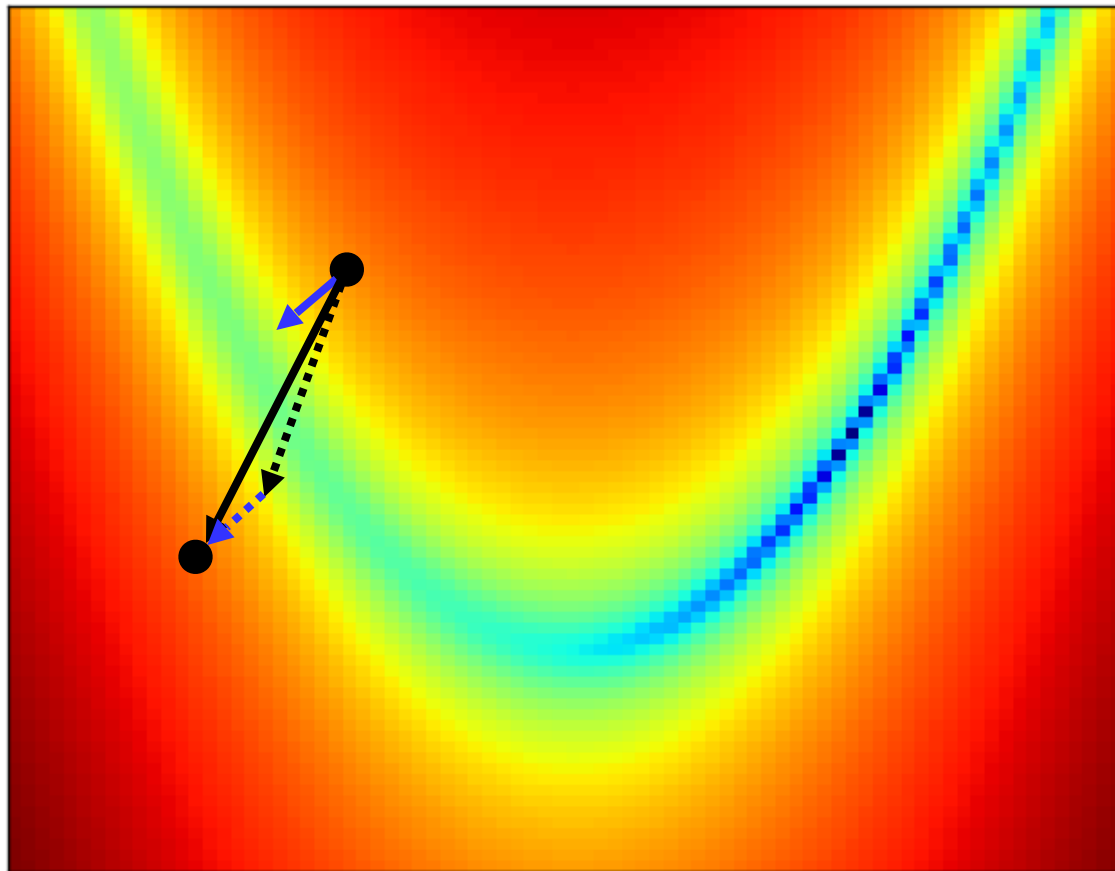
Metoda Nesterova

Klasyczna metoda pędu

$$-\epsilon \nabla_{\phi_t} J \quad \begin{array}{c} \text{solid blue arrow} \\ \text{dashed blue arrow} \end{array}$$

$$\mu \mathbf{M}_{t-1} \quad \text{dashed black arrow}$$

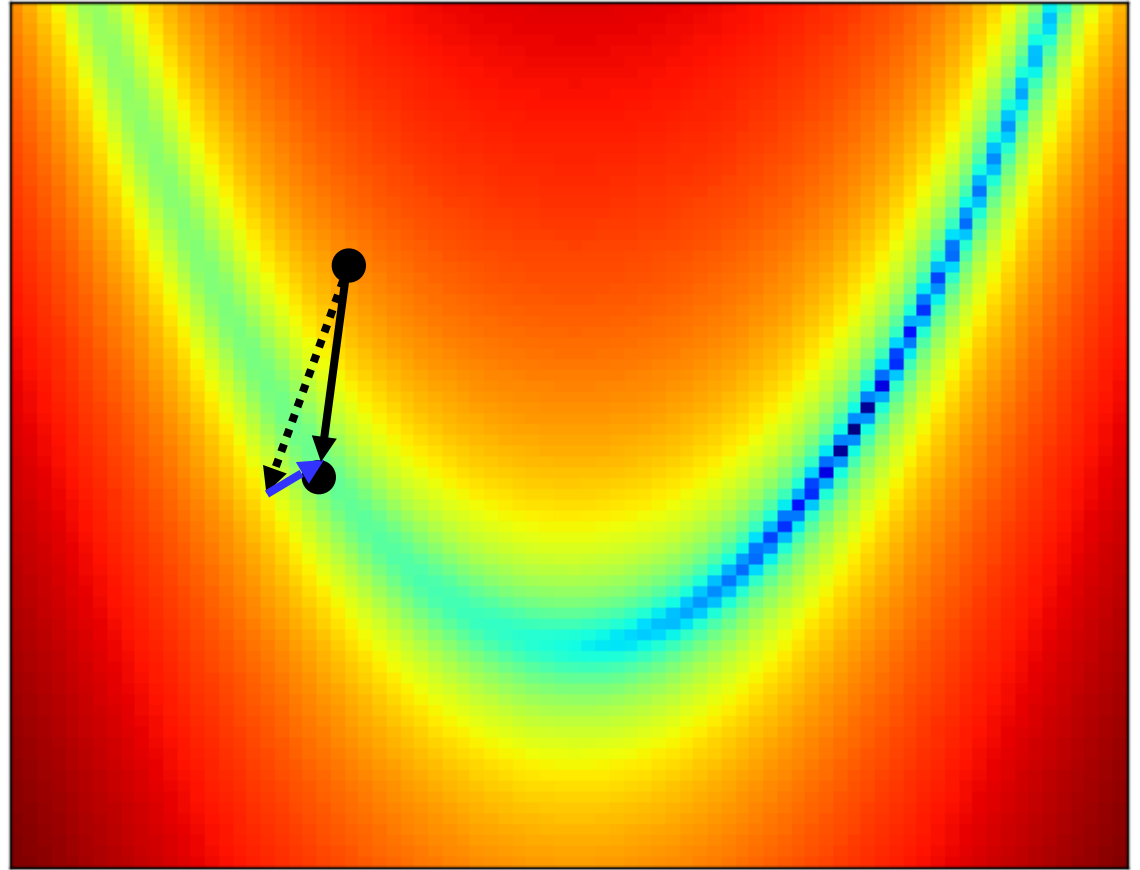
$$\mathbf{M}_t \quad \text{solid black arrow}$$



Metoda Nesterova

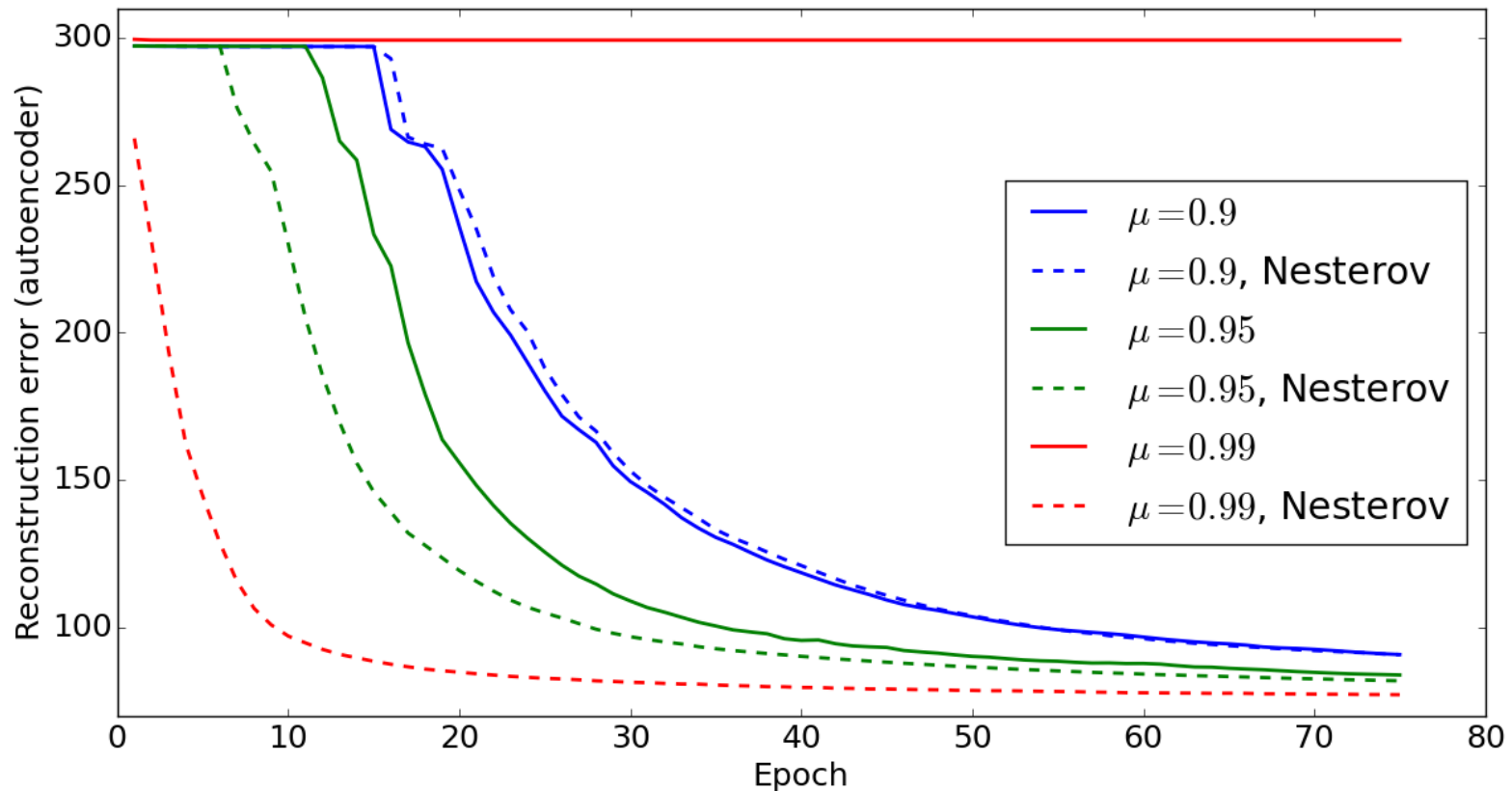
Metoda Nesterova

$$\begin{aligned} -\epsilon \nabla_{\phi_t + \mu \mathbf{M}_{t-1}} J &\longrightarrow \\ \mu \mathbf{M}_{t-1} &\dashrightarrow \\ \mathbf{M}_t &\longrightarrow \end{aligned}$$



Metoda Nesterova

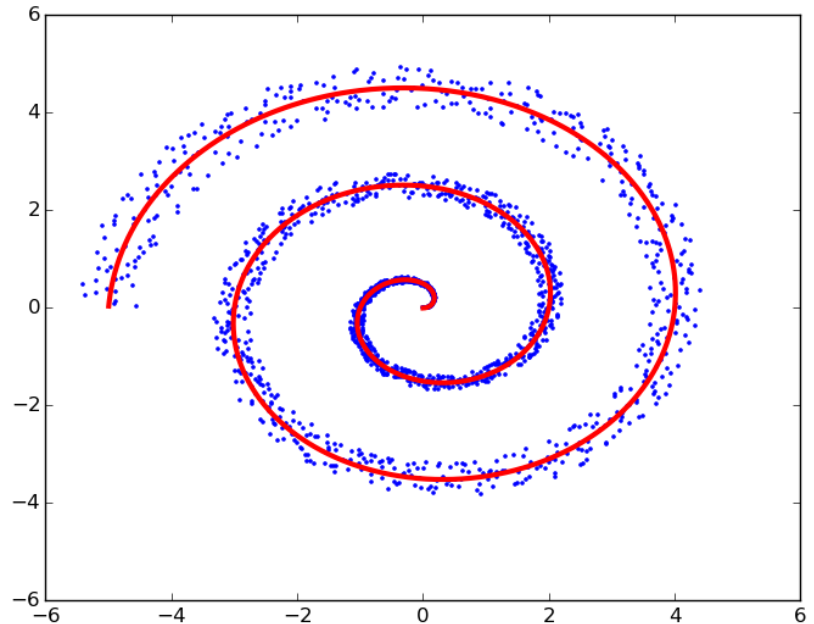
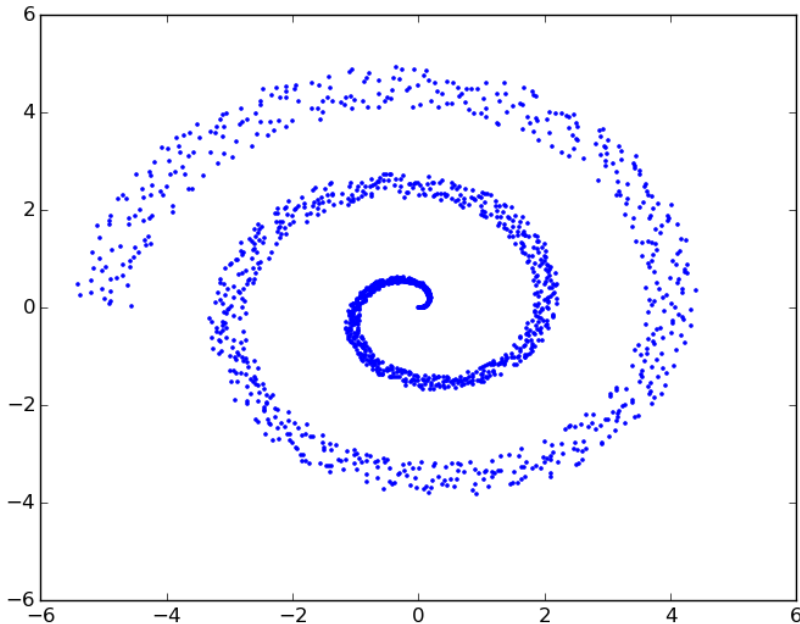
Metoda Nesterova pozwala optymalizować sieci neuronowe z bardzo dużymi wartościami pędu.



Sieci autoasocjacyjne (autokodery)

Redukcja wymiarowości

W wielu przypadkach liczba **znaczących wymiarów** w zbiorze danych jest mniejsza od jego pozornej wymiarowości.



Redukcja wymiarowości

W wielu przypadkach liczba znaczących wymiarów w zbiorze danych jest mniejsza od jego pozornej wymiarowości.

Na przykład:

- ❑ Cyfra w MNIST ma 784 piksele.
- ❑ „Pozorna” liczba wymiarów w tym zbiorze danych wynosi więc 784 wymiary.
- ❑ Jednak zbiór ten składa się jedynie z 10 cyfr, pisanych na wiele sposobów.
- ❑ Możemy więc przypuszczać, że „wewnętrzna” wymiarowość (ang. *intrinsic dimensionality*) tego zbioru danych będzie bliższa liczbie dziesięciu – kilkunastu wymiarów.

Redukcja wymiarowości

W wielu przypadkach liczba znaczących wymiarów w zbiorze danych jest mniejsza od jego pozornej wymiarowości.

Można wówczas przyjąć, że dane leżą na niskowymiarowej powierzchni, która jest „zanurzona” w przestrzeni wielowymiarowej.

Jednym z celów uczenia nienadzorowanego jest odkrycie poprawnej niskowymiarowej reprezentacji (pozornie) wielowymiarowego zbioru danych.

- ❑ Redukcja wymiarowości (ang. *Dimensionality Reduction*).
- ❑ *Manifold Learning*.

Redukcja wymiarowości a sieci neuronowe

Od dawna znane są liniowe metody redukcji wymiarowości, takie jak np. analiza składowych głównych (*Principal Component Analysis – PCA*).

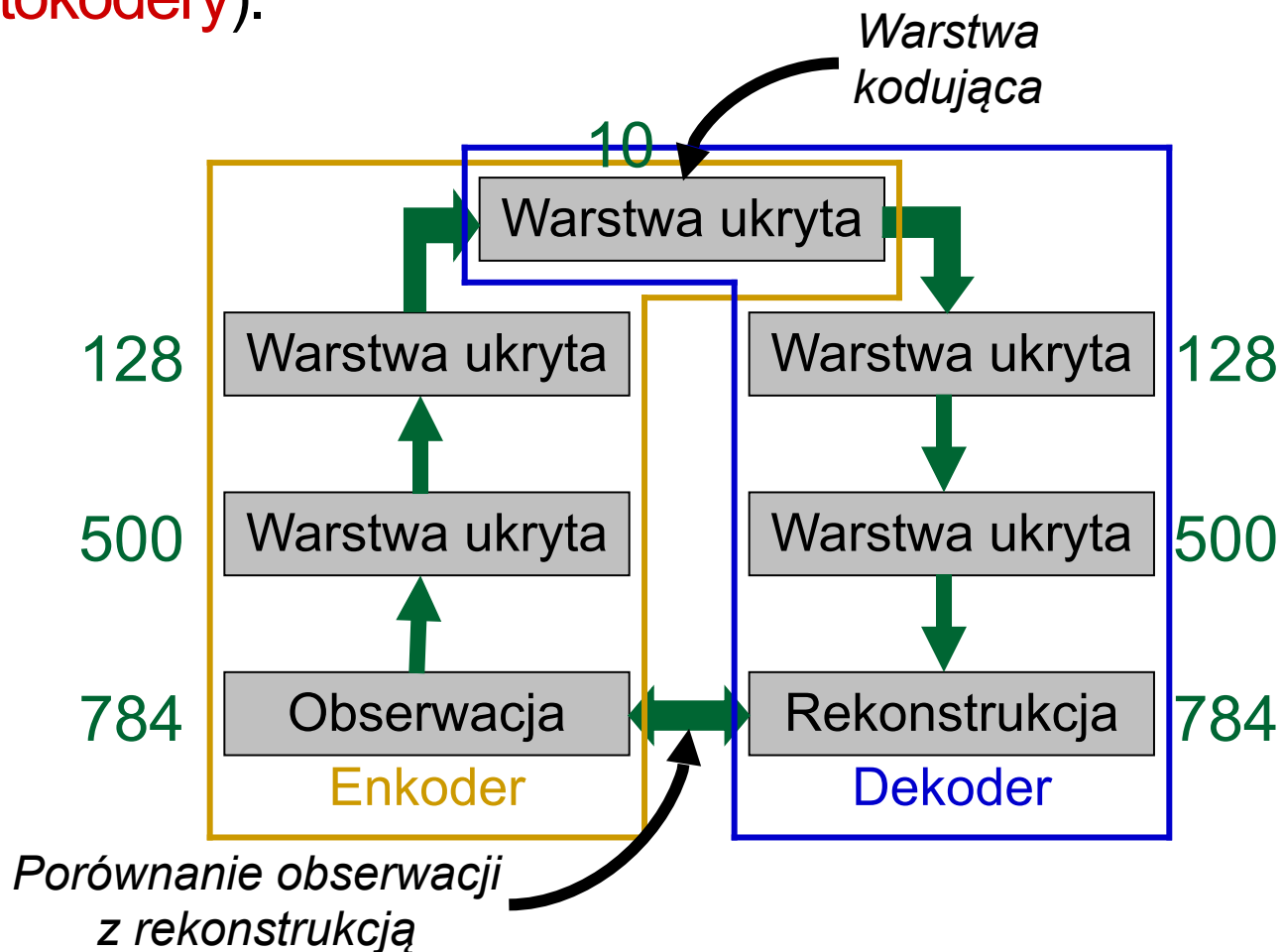
- ❑ Wybierz kilka prostopadłych osi w przestrzeni wejściowej.
- ❑ Oś wybierz tak, by rozrzut danych wzdłuż wybranych osi był jak największy.

Metody te nie są jednak w stanie odkryć powierzchni innych, niż hiperpłaszczyzny.

Okazuje się, że sieci neuronowe można wykorzystać do *nieliniowej redukcji wymiarowości*.

Sieci autoasocjacyjne

W tym celu można wykorzystać **sieci autoasocjacyjne** (**autokodery**).



Sieci autoasocjacyjne

Sieć autoasocjacyjną uczymy wsteczną propagacją błędu.

Funkcją kosztu jest błąd, z jakim zrekonstruowana została wejściowa obserwacja:

$$J(\mathbf{r}, \mathbf{v})$$

$\mathbf{v}$ – obserwacja

$\mathbf{r}$ – rekonstrukcja obserwacji

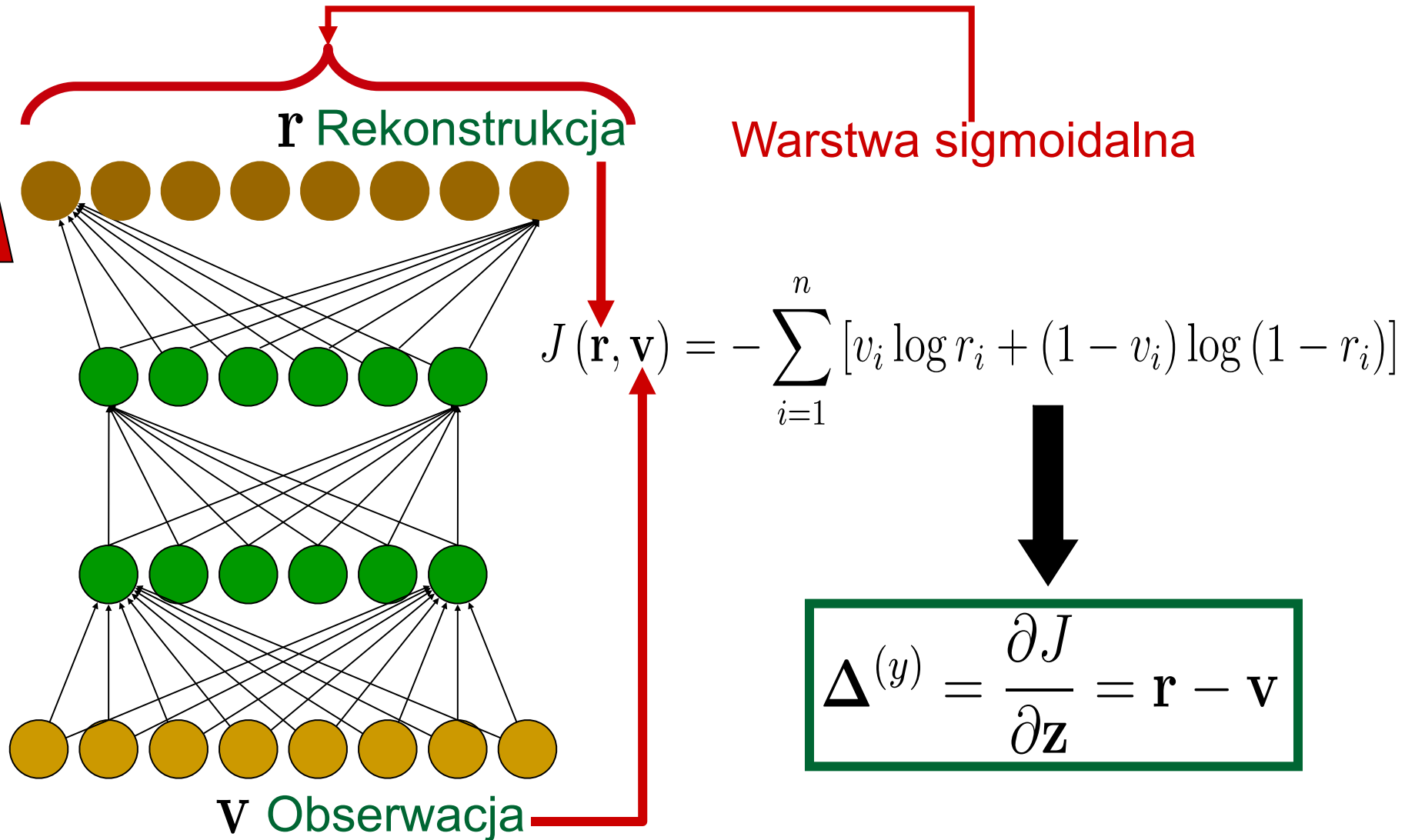
W tym przypadku algorytm uczenia nadzorowanego jest więc zaadoptowany do typowego zagadnienia uczenia nienadzorowanego.

Funkcja kosztu w sieci autoasocjacyjnej

Jaką postać może mieć funkcja kosztu?

- ❑ Funkcja kosztu zależy od rodzaju jednostek, którymi modelowane są dane.
- ❑ Jeśli wyście sieci (a więc również obserwacje) modelowane jest jednostkami sigmoidalnymi, każdy element obserwacji możemy potraktować jako prawdopodobieństwo, że odpowiadający mu neuron ma wartość 1.
- ❑ Z wykładu wiemy, że odpowiednią funkcją kosztu będzie wówczas (zanegowana) entropia krzyżowa.

Funkcja kosztu w sieci autoasocjacyjnej



Funkcja kosztu w sieci autoasocjacyjnej

Co w przypadku, gdy obserwacje nie dają się modelować jednostkami sigmoidalnymi?

W niektórych zastosowaniach obserwacje możemy przekształcić tak, by reprezentowały prawdopodobieństwa. Wówczas funkcją kosztu będzie oczywiście entropia krzyżowa.

- Możemy tak postąpić na przykład z obserwacjami reprezentującymi dokumenty w postaci wektorów zliczeń występujących w nich wyrazów.

W ostateczności jako funkcję kosztu możemy również przyjąć błąd kwadratowy:

$$J(\mathbf{r}, \mathbf{v}) = \frac{1}{2} \|\mathbf{r} - \mathbf{v}\|^2$$

Uczenie sieci autoasocjacyjnej

Architektura sieci autoasocjacyjnej sugeruje, że sieć ta będzie miała dużo warstw ukrytych.

- W jednej sieci łączymy enkoder i dekodery.

Jak wyuczyć tak głęboką sieć neuronową, by prawidłowo kodowała i rekonstruowała obserwacje?

- Koncepcja głębokich sieci autoasocjacyjnych pojawiła się po odkryciu wstecznej propagacji błędów. Jednak uczenie głębokich sieci autoasocjacyjnych nie dawało wówczas dobrych rezultatów.

Skuteczne uczenie głębokich sieci autoasocjacyjnych stało się możliwe po odkryciu zachłannego pretrenowania warstw ukrytych.

Uczenie sieci autoasocjacyjnej

***G.E. Hinton, R.R. Salakhutdinov, "Reducing the dimensionality of data with neural networks",
Science, 313(5786), 504–507, 2006.***

Zauważ, że sieć RBM potrafi rekonstruować obserwację na podstawie stanu warstwy ukrytej.

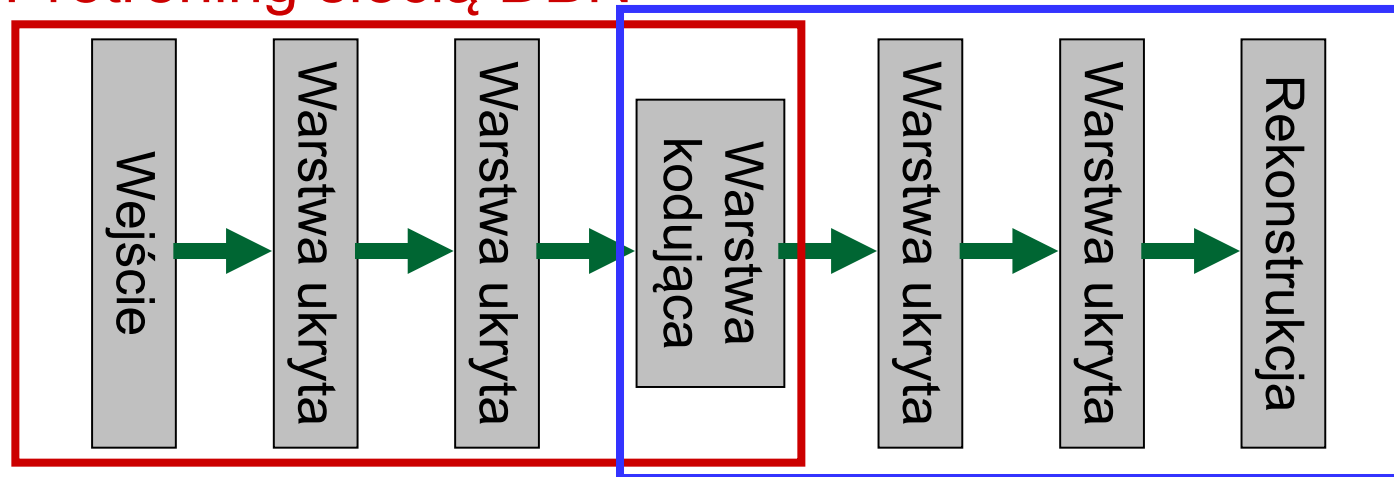
Tak więc, aby zainicjalizować głęboką sieć autoasocjacyjną:

- ❑ Wykonaj pretrening enkodera za pomocą sieci DBN.
- ❑ Dekoder zbuduj z tego samego DBN, odwracając kolejność warstw i transponując macierze wag.

Uczenie sieci autoasocjacyjnej

***G.E. Hinton, R.R. Salakhutdinov, "Reducing the dimensionality of data with neural networks",
Science, 313(5786), 504–507, 2006.***

Pretrening siecią DBN

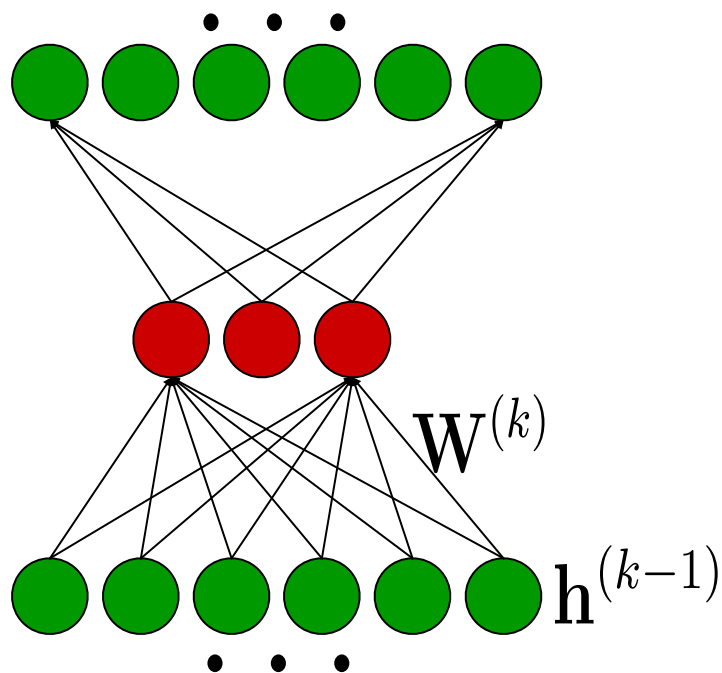


Transponowany enkoder¹

¹ Odwrócona kolejność warstw i transponowane macierze wag.

Warstwa kodująca

Jaka funkcja aktywacji będzie odpowiednia dla warstwy kodującej?



- ❑ Warstwa kodująca wykonuje projekcję sygnału na niskowymiarową powierzchnię.
- ❑ Projekcję chcemy z reguły otrzymać w postaci wektora liczb rzeczywistych.
- ❑ Dobrą funkcją aktywacji będzie w tym przypadku aktywacja liniowa:

$$f(z) = z \quad \left(= \mathbf{h}^{(k-1)} \mathbf{W}^{(k)} \right)$$

RBM z jednostkami Gaussowskimi

Ale jak wykonać pretrening warstwy z liniową funkcją aktywacji?

- ❑ Klasyczny RBM zakłada, że jednostki są binarne.

Musimy zbudować model opisujący wartości rzeczywiste w warstwie ukrytej.

- ❑ Modelem takim może być RBM z jednostkami Gaussowskimi w warstwie ukrytej.

RBM z jednostkami Gaussowskimi

RBM z Gaussowskimi jednostkami ukrytymi różni się od klasycznej sieci RBM funkcją energii.

Zakładając binarne jednostki widoczne, funkcję energii w tym modelu definiujemy jako:

$$E(\mathbf{v}, \mathbf{h}) = \sum_{j=1}^m \frac{(h_j - c_j)^2}{2s_j^2} - \sum_{i=1}^n \sum_{j=1}^m v_i w_{ij} \frac{h_j}{s_j} - \sum_{i=1}^n v_i b_i$$

gdzie:

b_i – biasy warstwy widocznej

c_i – biasy warstwy ukrytej

w_{ij} – wagi

RBM z jednostkami Gaussowskimi

Aktywacja jednostki ukrytej jest wówczas zmienną losową z rozkładu Gaussa o wariancji równej s_i^2 .

- ❑ W procesie uczenia musimy więc ustalić nie tylko wagi i biasy, lecz również wariancję każdej jednostki ukrytej.
- ❑ Uczenie się wariancji dla jednostek ukrytych jest w praktyce dosyć trudne.
- ❑ Model ten jest więc z często upraszczany poprzez założenie, że każdy neuron w warstwie ukrytej ma jednostkową wariancję.

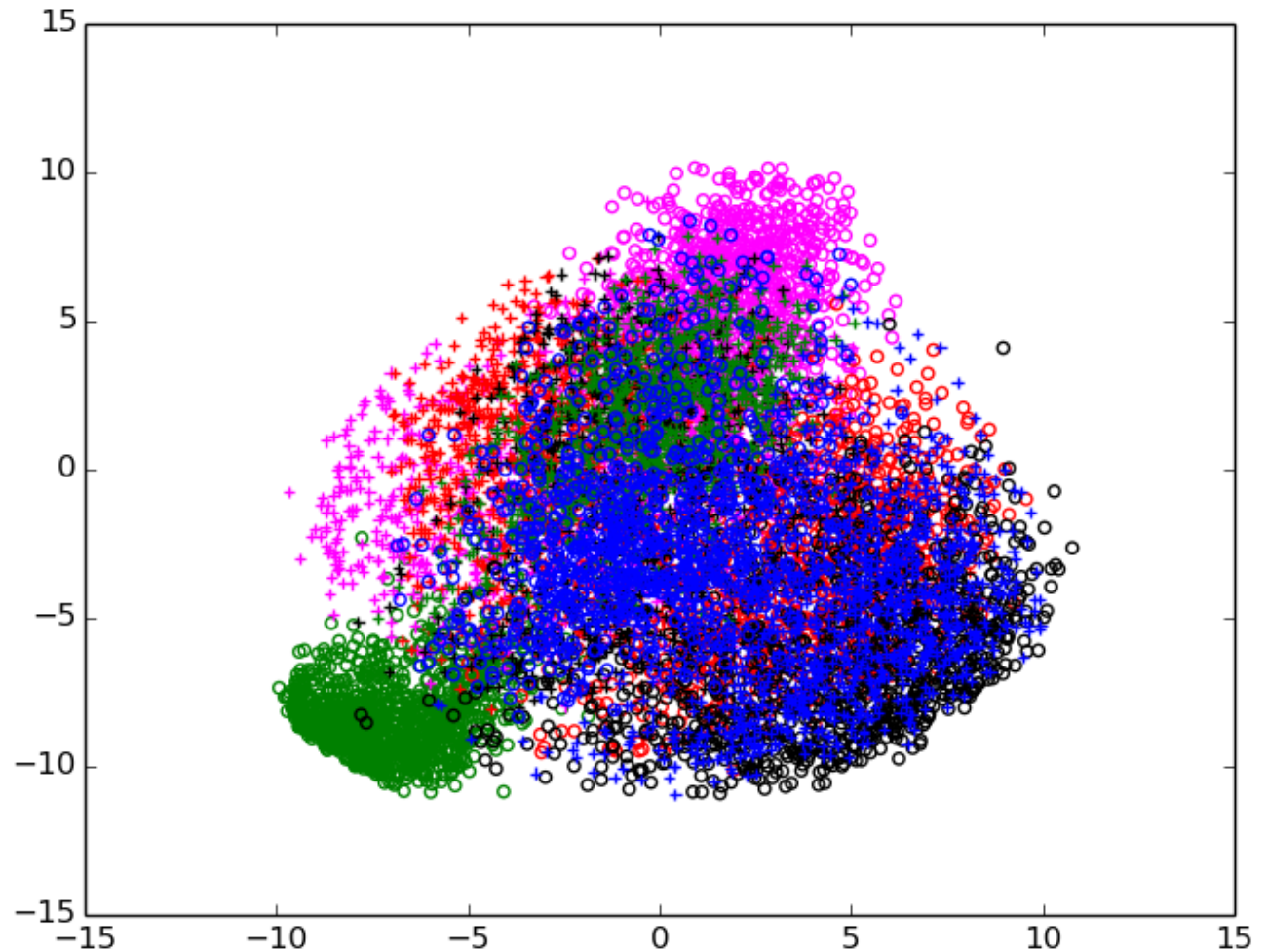
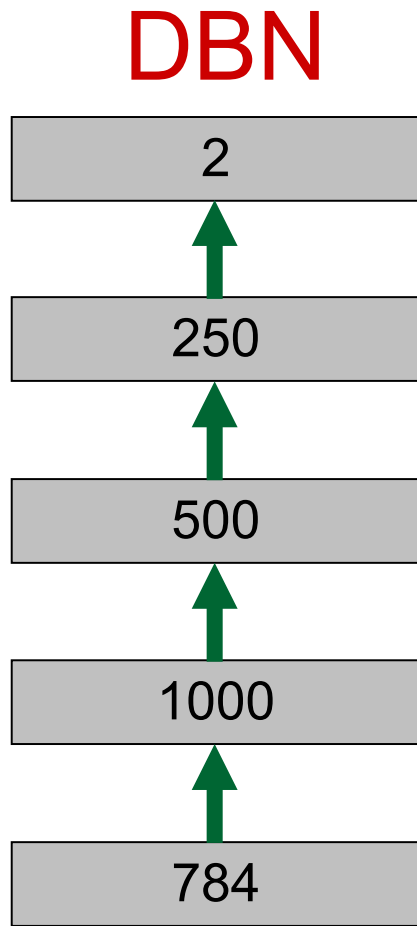
RBM z jednostkami Gaussowskimi

Zakładając jednostkową wariancję dla każdej jednostki ukrytej otrzymujemy model, w którym schemat CD-k jest taki sam jak w klasycznej sieci RBM.

Każdy neuron w warstwie ukrytej jest wówczas zmienną losową z rozkładu Gaussa o jednostkowej wariancji i wartości przeciętnej równej całkowitemu wejściu do tego neuronu:

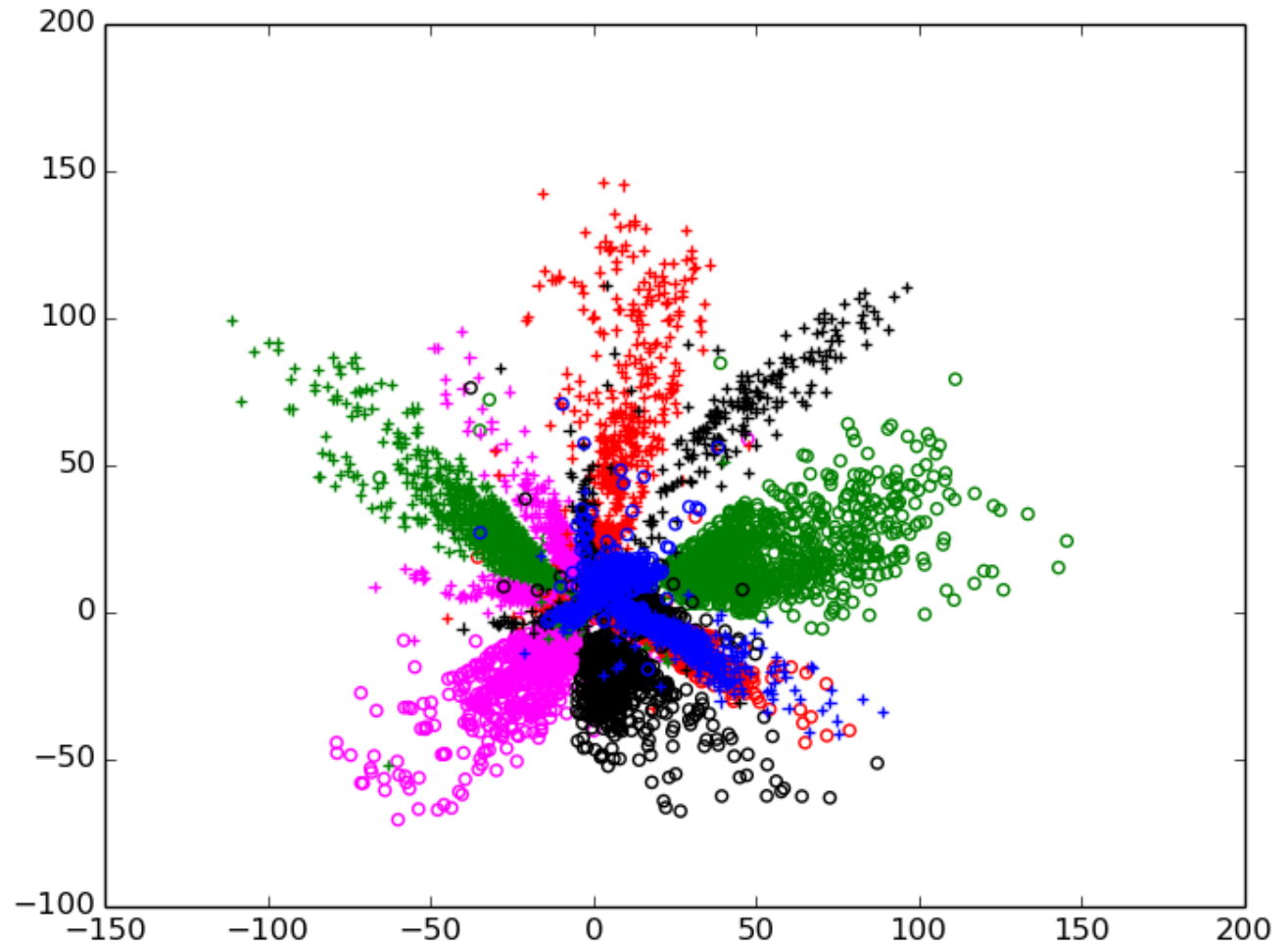
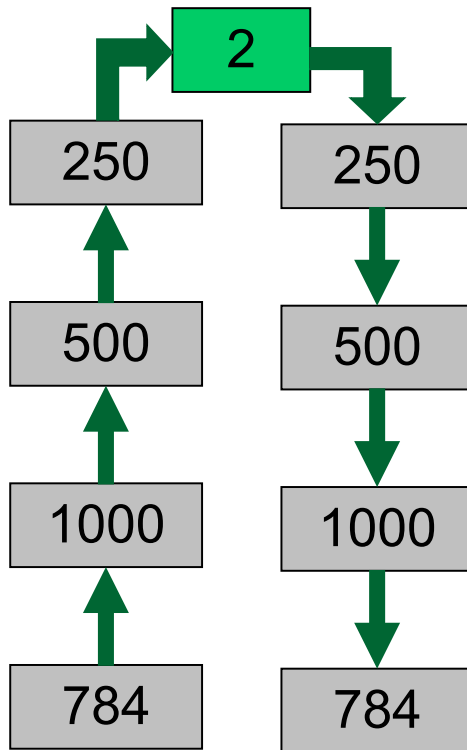
$$h_j \sim N(\mathbf{v}\mathbf{W}_{-j}, 1)$$

Przykład - wizualizacja zbioru MNIST



Przykład - wizualizacja zbioru MNIST

Autokoder



Dygresja: RBM – Gaussowska warstwa widoczna

RBM może również modelować wartości rzeczywiste w warstwie widocznej.

Wykorzystamy wówczas **RBM z Gaussowskimi jednostkami widocznymi**:

- Każdy neuron w warstwie widocznej jest modelowany jako zmienna losowa z rozkładu Gaussa o jednostkowej wariancji i wartości przeciętnej równej całkowitemu wejściu do tego neuronu:

$$v_i \sim N(\mathbf{h} \mathbf{W}_{i,-}^T, 1)$$

- Warstwę ukrytą modeluje się wówczas jako jednostki binarne lub jednostki ReLU.
- Również w tym modelu schemat CD-k nie ulega zmianie (w stosunku do klasycznego RBMa).

Dygresja: RBM – Gaussowska warstwa widoczna

RBM z Gaussowskimi jednostkami widocznymi wymaga odpowiedniego przygotowania danych wejściowych.

Każdy wymiar obserwacji należy tak znormalizować, by jego wartość przeciętna na zbiorze **trenującym** wynosiła 0 a odchylenie standardowe (na zbiorze trenującym) było jednostkowe:

$$\mu_i = \frac{1}{\#\mathcal{X}} \sum_{\mathbf{v} \in \mathcal{X}} v_i$$

$$s_i = \sqrt{\frac{1}{\#\mathcal{X}} \sum_{\mathbf{v} \in \mathcal{X}} (v_i - \mu_i)^2}$$

$$v_i \leftarrow \frac{v_i - \mu_i}{s_i}$$

$\mathcal{X}$ – zbiór trenujący

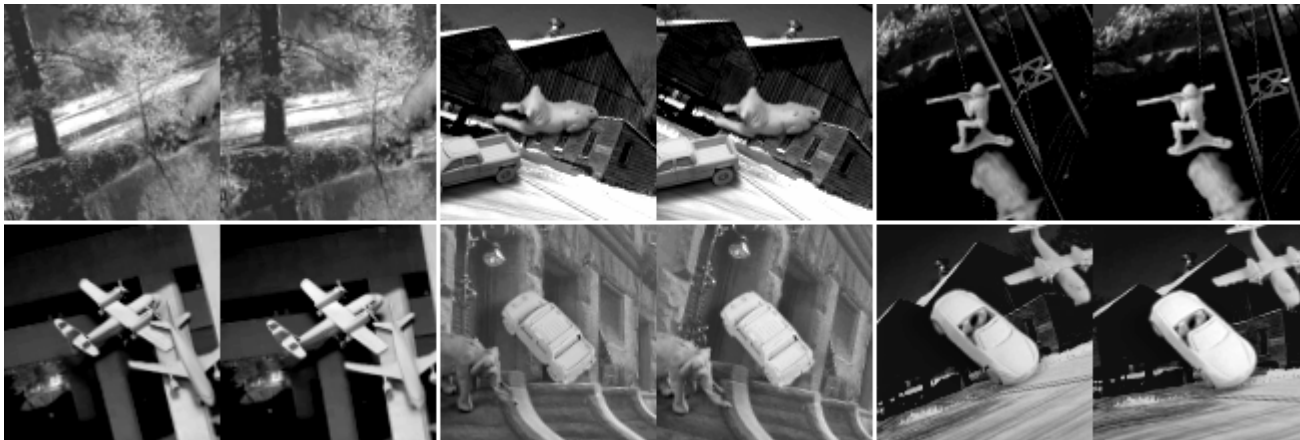
Dygresja: RBM – Gaussowska warstwa widoczna

Jakie cechy potrafi wykryć RBM z Gaussowskimi jednostkami widocznymi?

Przykład:

V. Nair, G.E. Hinton, "Rectified linear units improve restricted boltzmann machines", In: Proc. 27th International Conference on Machine Learning (2010)

- ❑ Zbiór Jittered-cluttered NORB.
- ❑ RBM z jednostkami Gaussowskimi w warstwie widocznej (z jednostkową wariancją) i ReLU w warstwie ukrytej.



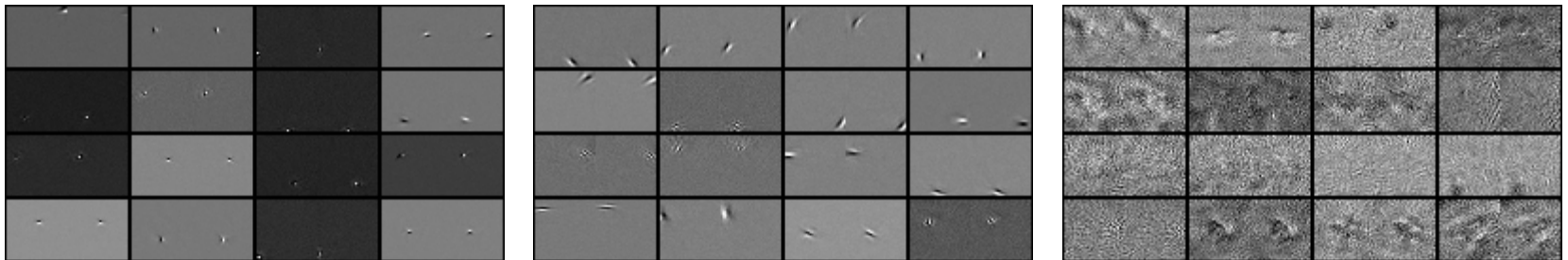
Dygresja: RBM – Gaussowska warstwa widoczna

Jakie cechy potrafi wykryć RBM z Gaussowskimi jednostkami widocznymi?

Przykład:

V. Nair, G.E. Hinton, "Rectified linear units improve restricted boltzmann machines", In: Proc. 27th International Conference on Machine Learning (2010)

- ❑ Zbiór Jittered-cluttered NORB.
- ❑ RBM z jednostkami Gaussowskimi w warstwie widocznej (z jednostkową wariancją) i ReLU w warstwie ukrytej.



Kolejny wykład

Sieci konwolucyjne
